Meme Sniper: Strategy Audit and Implementation Plan

Application: BTC Paper Trading Lab, Meme Sniper v10.1
Reviewed artifact: `btc-paper-lab-site.zip`, containing `index.html` (identical to the supplied `lab.html`)
Review date: October 21, 2026
Review type: Static source-code audit, strategy design review, and external documentation research
Scope: Meme Sniper (`SNIPER`, `SNIPER_CORE`, `SNIPER_UI`) only. The separate daily Meme Coin Trader and hourly Solana Meme Trader are different strategies and are not the subject of this report.

> **Bottom line:** The existing system is a thoughtful first-generation experiment, but its apparent precision is misleading. The largest issue is not simply that it buys dips or waits too long to sell. It is that the paper engine can act on information that would not have been available when a trade supposedly happened. Fix timing and execution credibility first. Then replace the strategy's single-trigger mindset with independent entry, ongoing thesis validation, and exit decisions. Social research should improve those decisions, but it must be verifiable, timestamped, and tested to determine whether it actually contributes predictive value.

Executive assessment

Current strategy as implemented: Monitor recently tradeable SOL-paired pump.fun tokens, follow them to approximately $7,000 market capitalization, wait for security checks and a token-name-based meme match, add qualifying tokens to up to six watchlist slots, buy one SOL on the first qualifying 40% pullback from the launch high, and exit at 1% net profit, a 90% spot-price loss, or a variant-specific time limit. Seven variants start with separate 10 SOL paper accounts. Source: `index.html` lines 3542-3732 and 3733-4268.

Important strengths worth keeping: There is a shared browser/server strategy core, explicit paper-only boundaries, separate bots for controlled comparisons, meaningful token-audit inputs, a fees-and-price-impact model, a `rearm` provision intended to avoid buying a token that was already below the trigger at watch initiation, candle finality checks, and trade journaling. Those are genuine positives. They do not solve the issues below.

Most urgent findings, in order:

1. **P0, potential look-ahead bias:** Tokens become eligible after asynchronous checks, but the trade watch begins at the earlier crossing timestamp. This can create retroactively booked buys.
2. **P0, non-executable paper fills:** The system books trigger-based en-

tries and exits using historical one-minute OHLC prices, including prices that were never observed live at decision time. Orders are not actually submitted, and some entry variants fill at the candle close that established their own signal.

3. **P0, broken practical downside protection:** The main bot uses a 90% stop with no maximum holding period. For low-cap coins, that stop can be below the modeled bonding-curve floor, meaning it cannot trigger while on the curve.
4. **P0, stale or synthetic valuation:** If sufficiently fresh price and candle data disappear, an open position may be marked using an old price or even its entry price, rather than being flagged as unpriceable.
5. **P1, weak entry intelligence:** The primary signal is a large decline, not evidence of recovering demand. Candidate selection chooses the earliest qualifying trigger, not the best current opportunity.
6. **P1, weak exit intelligence:** There are no independent momentum, liquidity, holder-selling, or social-attention deterioration exits. The code collects several useful fields but does not use them in trade decisions.
7. **P1, artificial hype proxy:** The mandatory meme test is a shared word among names of other recently crossing tokens, not a measurement of attention, trust, or community participation.
8. **P1, incomplete security interpretation:** Missing developer concentration is treated as 0%, and missing top-holder concentration passes the default check.
9. **P1, execution-model mismatches:** Pool reserves, fee tiers, network costs, route quality, and latency cannot be inferred reliably from a candle alone. Some constants may diverge from current protocol settings.
10. **P1, insufficient evaluation:** Trade count, realized-only drawdown, and raw balance comparisons can flatter a system holding large underwater or untradeable positions. No verified independent historical Sniper performance is present in the uploaded artifact.

These are **code-supported findings**, not proof of historical dollar losses. The uploaded website does not include the deployed GitHub Actions code, the current server journal, a longitudinal candidate data archive, or executable transaction receipts. I have **not** independently run a real-time paper trial or a point-in-time historical backtest. Numerical examples below are illustrations using the source code's mathematics, not claims about completed trades.

A. The current decision chain, and why it matters

1. `discover()` requests Jupiter's recent tokens and follows pump.fun listings that appeared within the prior 75 minutes. `follow()` polls DexScreener and checks for the first observed $7,000 crossing. (`index.html` lines 3860-3903.)
2. `onCross()` records the crossing time in `k.x`, adds the coin to a common crossing history, and queues Jupiter and RugCheck lookups. (`index.html`

lines 3904-3913.)

3. `checkLoop()` and `rcLoop()` gather audits and token fields asynchronously. `decide()` requires both inputs before accepting a candidate. (`index.html` lines 3934-4006.)
4. The basic checks require revoked mint and freeze authorities, maximum 10% developer holding, maximum 30% top-holder holding when provided, no RugCheck `danger` flag, and at least one name-word match to another $7,000-crossing token within two hours. (`index.html` lines 3960-3987.)
5. A passing coin enters a two-hour watchlist. `findEntry()` looks for a 40% drop from a prior high. The standard paper purchase occurs at the trigger or better if the minute's low touched it. (`index.html` lines 3585-3624, 3987-4013.)
6. `hold()` checks stop, take-profit, then optional elapsed-time exit for each finalized candle. The normal strategy has no maximum hold. (`index.html` lines 3626-3652.)
7. `stepBot()` searches watched coins and selects the one with the earliest recorded qualifying entry minute. The position then occupies the bot until an exit is recorded. (`index.html` lines 4139-4183.)
8. Paper value is estimated by a constant-product reserve model and current or fallback market price; performance uses realized trades plus estimated value of one open position. (`index.html` lines 3557-3578, 4184-4200.)

Architectural conclusion: The program is primarily a **post-event price-trigger replay system**, with a set of eligibility filters. It is not yet a system that maintains and tests an investment thesis throughout the lifetime of a position.

B. Detailed defects, fixes, and acceptance tests

Finding 01. Retroactive eligibility and backdated entries

Severity: P0. **Code evidence:** `onCross()` sets `k.x` at crossing, while `decide()` later inserts `{from: k.x, to: k.x + WATCH_S}` regardless of the completion time of Jupiter and RugCheck verification. See lines 3904-3913 and 3981-4004.

Problem: Consider a token crossing the threshold at 12:00. RugCheck finishes at 12:05. The bot accepts the token at 12:05 but is allowed to search for a purchase from 12:00 onward. If its dip happened at 12:02, the dashboard can book an entry at 12:02 even though the bot did not yet have completed checks and could not have authorized that entry then. A subsequent favorable price movement can make this look especially profitable.

Required change: Store both `cross_observed_at` and `all_checks_available_at` with monotonic wall-clock timestamps. Make `eligible_at = max(cross_observed_at, all_checks_available_at, strategy_ready_at)`. Signal observation and simulated order submission may occur **no earlier** than `eligible_at`. Do not

silently backdate. For historic retrospective replay, tag hypothetical entries separately from forward-observed paper decisions.

Implementation steps: Add point-in-time metadata to every audit response and risk input; set `watch.from = eligible_at`; store the exact checks, values, timestamps, and API response IDs used in approval; maintain a decision event log. If a check arrived too late, record `missed_opportunity` rather than pretending a trade occurred. Require all entry indicators to have been known at signal time.

Acceptance test: Cross at 12:00, checks complete 12:05, price touches buy trigger 12:02, recovers by 12:04. Expected: zero forward-paper buys. A fresh signal at 12:06 may become eligible. This test must fail with the old implementation and pass with the new one.

Finding 02. OHLC touch does not prove executable entry

Severity: P0. **Code evidence:** `findEntry()` treats a candle low below the trigger as a completed fill: `if (l <= trig) fillUsd = Math.min(o, trig)`. For the green entry, it uses the current candle close after inspecting the candle's low and whether it closed green. See lines 3606-3613.

Problem: The code assumes a fill because a historical bar touched a price. A single wick on thin liquidity does not mean a one-SOL swap could have executed there. The green-candle alternative is especially problematic: it recognizes a candle as green only after that candle completes, yet fills at that same candle's closing price. Neither modeled slippage nor conservative stop ordering fixes this timing problem.

Required change: Separate **signal time**, **submission time**, **first executable time**, and **confirmed fill time**. A strategy using completed candles must trade on the **next eligible quote or event**, after latency and fees, not the just-completed candle close. If event-level quotes are unavailable, use a conservative subsequent-candle method with explicit uncertainty bounds; do not label those fills as observed live executions.

Acceptance tests: Same-candle green signal cannot fill at that candle close; an isolated low wick with no quote or subsequent trade may result in a missed order; a delayed fill reflects a changed price. Every trade stores the observation available to the bot before the simulated submission.

Finding 03. Stop loss may be structurally unreachable

Severity: P0. **Code evidence:** `DEF.sl = 0.90`, `DEF.maxHold = 0`, `CURVE_FLOOR = 30 / 1.073e9`. The UI explicitly warns when the stop is below the curve floor and says the unrestricted bot then waits for profitability. See lines 3579, 3702-3706, 4820-4829 and 4944-4952.

Problem: The stop sits at just 10% of entry price. At an illustrative SOL price

of US$150, the code's modeled floor is approximately US$4,194 market cap. A stop below this floor is unreachable for a bonding-curve entry below approximately US$41,940 market cap. A token can be almost abandoned without ever triggering the configured stop. The main bot has no time limit, so capital can remain tied up indefinitely.

Required change: Replace the 90% stop with a layered, executable exit policy. Use risk budget per position, market-structure invalidation, time-to-live, poor-liquidity response, and fast deterioration exits. For all price-stop strategies, test the proposed stop against the actual venue's achievable price range and expected executable proceeds. Declare an invalid stop configuration **before** entry, instead of merely warning about it after entry.

Acceptance tests: Any proposed entry with a mathematically unreachable protective level is rejected or converted to a realistic exit plan. A coin that falls to the curve floor and ceases trading is marked distressed, not held forever with a fictional stop. A dead position's recoverable value and opportunity cost are visible.

Finding 04. No independent 'coin is dying' exit

Severity: P1. **Code evidence:** `hold()` has only `sl`, `tp`, `time`, and `horizon/open`. `stepBot()` closes only when `hold()` returns `done`. See lines 3626-3652 and 4139-4155.

Problem: Deterioration can become evident long before either fixed price boundary is touched. Examples: buyers disappear; distinct buyers decline; sell value dominates buys; repeated bounce attempts fail; top wallets distribute; liquidity shrinks; social conversation evaporates. None of these can independently trigger an exit in the current code.

Required change: Build a **thesis-invalidation service** that evaluates every held coin on each available event. Use multiple indicators and timeframes, weighted by data quality, with an emergency override for verified catastrophic threats. A single red candle alone should not be enough. Exit urgency must increase when sell-route quality deteriorates, even if an ideal spot price has not fallen to a stop.

Recommended first deterministic model, explicitly experimental: Maintain 1-minute, 3-minute, and 5-minute price and flow windows. Define deterioration features such as drawdown from recent valid high, sequence of failed highs, negative net SOL flow, decrease in distinct buyers, ratio of seller value to buyer value, price recovery failure, and liquidity/quote deterioration. Require corroboration from two independent feature families to avoid noise, except for an emergency safety event. Test multiple thresholds without declaring any preset combination optimal.

Acceptance tests: Persistent weakening with corroborating order-flow evidence exits before a 90% decline; one isolated red candle does not force exit;

a confirmed loss of a usable sell route creates an explicit incident state and a realistic valuation.

Finding 05. Trade management is blind to the thesis after entry

Severity: P1. **Code evidence:** `decide()` runs once during candidate approval; `hold()` only takes candles, SOL price, position, and static parameters. No new social, audit, or holder inputs are consulted. See lines 3980-4019 and 3626-3652.

Problem: A coin could pass security checks at 12:00 and suffer a dangerous wallet event at 12:08. The bot's entry thesis and risk clearance are never reconsidered. Similarly, a genuinely viral coin might continue accelerating while a fixed target forces an early sale.

Required change: Each open position needs an explicit **entry thesis**, observable invalidation rules, and state transitions: `healthy`, `watching deterioration`, `thesis broken`, `exit pending`, `exit filled`, or `exit unavailable`. Refresh decision features and log why the thesis remains valid. Security changes outrank social hype. Store what changed since entry.

Acceptance tests: Inject post-entry developer selling, social collapse, improving liquidity, and sustained buying; verify appropriate state changes without changing historical decisions retroactively.

Finding 06. Mandatory 'meme' filter is not social intelligence

Severity: P1. **Code evidence:** `words()` strips token names and symbols into words; `memeScore()` counts other $7,000-crossing tokens with matching words within two hours. It contributes 10 points per matching token to candidate ranking. See lines 3677-3690, 3979-3980.

Problem: A group of copycat or spam tokens can make a word look popular without a single credible X post. An original viral coin with a distinctive name may fail because no other coin shares a word. This is not even a direct measure of community attention, much less proof of organic hype.

Required change: Rename this metric `name_cluster_score` and demote it to weak supporting evidence. Add a separate, clearly sourced `attention_evidence` component. Link token mint to purported X/Telegram/site identities; verify mentions of the exact mint when practical; distinguish official account claims from third-party commentary; measure independent speakers, change in attention, engagement quality, and recency. **Do not** allow an AI model to invent social observations or autonomously override hard safety gates.

Acceptance tests: Ten copycat names with no credible engagement do not qualify solely because of their words; a distinctive token with genuinely increasing independent attention can qualify if price, liquidity, and security evidence

also support it; unavailable social APIs result in `unknown`, not `no interest` or `verified hype`.

Finding 07. Useful market signals are collected but not consumed

Severity: P1. **Code evidence:** Jupiter's `stats5m` is stored in `k.s5`, DexScreener `txns.m5`, `volume.m5` in `r.m5`, and liquidity in `r.liq`. They are not referenced by `findEntry()` or `hold()`. See lines 3942-3949, 4092-4112.

Problem: The engine possesses partial evidence about buyers, sellers, volume, and liquidity but still acts almost entirely on percentage decline. Even the `organicScore` mainly informs a secondary watchlist score, not the trigger or exit.

Required change: Create a `MarketSnapshot` feature model with timestamp, venue, quote asset, buyers, sellers, unique traders where available, net volume, liquidity, price structure, organic score, and provenance. Derive rolling measures rather than using isolated absolute values. Normalize by market cap, token age, liquidity, and market regime. Distinguish transaction count from notional buy/sell amount, since ten dust buys may not offset one large sale.

Acceptance tests: Two tokens with the same dip but opposite organic buying patterns receive different eligibility decisions. Data that lacks buyer-versus-seller notional cannot pretend to measure net cash flow.

Finding 08. The bot purchases whichever qualifying dip occurred first

Severity: P1. **Code evidence:** `stepBot()` chooses the lowest `entryT` from qualifying watched coins. `scoreOf()` only decides which candidate occupies a limited watchlist slot. See lines 3981-4002 and 4156-4181.

Problem: The bot can take the earliest collapsing coin while a stronger one forms a slightly later confirmed setup. It optimizes chronological priority, not current expected trade quality. Selection must not be based on data that appeared after the chosen entry time, either.

Required change: Evaluate candidates **as of a common decision time**. Rank eligible, executable opportunities by expected post-cost return and downside, with security and liquidity gates applied first. If there is no adequately supported opportunity, hold cash. Include a cooldown after failed entries, and prevent repeated buys into one deteriorating coin.

Acceptance tests: Given a weak coin with an earlier raw dip and a stronger coin with later confirmed demand, the decision engine does not automatically choose the earliest one. It records why the selected coin was preferred and which alternatives were rejected.

Finding 09. Missing safety data can pass as safe

Severity: P1. **Code evidence:** `dev = a.devBalancePercentage == null ? 0 : +a.devBalancePercentage`, and top-holder test `top == null || top <= C.topMax`. See lines 3960-3973.

Problem: Unknown developer holdings appear as 0%, which is reported as satisfying the developer constraint. Unknown holder concentration passes the default condition. This is an epistemic error: **not reported is not the same as zero risk**. Some required checks properly fail closed, so this is an inconsistency rather than a universal problem.

Required change: Use four-valued security outcomes: `pass`, `fail`, `unknown`, `not_applicable`. Set explicit requirements for unknown critical fields. Re-query, cross-check a different provider, or refuse the entry. Keep risk signals separate from benign informational fields so an API outage does not silently become a green check.

Acceptance tests: Missing developer concentration does not print '0% held' or count as passing; missing top-holder data cannot bypass the risk budget; unverified API schema changes cause a visible validation error and safe rejection.

Finding 10. Risk checks and their UI timestamps do not tell the full truth

Severity: P1. **Code evidence:** The interface says checks were 'taken at' the crossing timestamp `k.x`; the actual audit requests occur afterward and their receipt timestamps are not stored. See lines 3934-3960 and 4850-4862.

Problem: The dashboard may imply that an audit was known earlier than it was. Without timestamps, the engine also cannot enforce data freshness or reproduce why a historical decision occurred.

Required change: Show `crossing observed`, `request sent`, `response received`, `effective as-of time`, and `last refreshed` separately. Preserve audit raw fields or hashes and source versions. Re-check security during open positions, prioritized by material risk changes.

Acceptance tests: A check completed at 12:05 cannot be labeled as performed at 12:00. A historical decision reproduces from preserved point-in-time fields without requesting today's revised audit.

Finding 11. Price monitoring is not trade-event monitoring

Severity: P1. **Code evidence:** DexScreener price polls normally run every 8 seconds, but entries/exits are decided from **finalized GeckoTerminal minute candles**. Candle requests run on a shared, rate-limited lane; held coins far from predefined exits can be scheduled every 240 seconds. See lines 3758-3782, 4121-4137, 4201-4216 and 4497-4515.

Problem: A 10-second liquidity event may not be acted on until minutes later, yet the replay may show a fill at a historical threshold. The `nearExit()` acceleration depends on proximity to the original TP or huge 90% stop, not independently on fast deterioration. Furthermore, DexScreener's last reported spot price is not a confirmed executable quote at the present moment.

Required change: Separate the low-frequency dashboard feed from the safety-critical event and quote feed. For new coins, favor a low-latency on-chain transaction, pool reserve, or authoritative quote stream if sustainably available. If data are only minute bars, deliberately **downgrade the supported strategy speed** and never claim true instant sniping. Give held positions and exit attempts priority over scanning new candidates.

Acceptance tests: A sudden liquidity event raises a safety alert from the fastest available source. A missing or delayed data feed pauses new entries. A completed-bar strategy cannot claim sub-minute fills.

Finding 12. Stale pricing can hide losses

Severity: P0. **Code evidence:** `markOf()` uses a price from the last fresh DexScreener observation, else the last retained candle, else the position's entry price. `solAt()` can look backward up to 240 minutes, then **forward** up to 239 minutes, finally falling back to current SOL or a hardcoded 100. See lines 3819-3829 and 4185-4200.

Problem: A dead coin with no current quote may display an appealing, stale value. In historical reconstruction, looking forward for a missing SOL conversion can leak future exchange rates. `solReady()` can allow a delayed event even when the needed exact SOL candle is missing. The interface uses the language 'if sold right now' even when the input price may not be fresh.

Required change: Implement strict freshness limits and an explicit `UNPRICEABLE` state. Track `last_trade_age`, `last_quote_age`, `last_pool_update_age`, and last independently verified sell route. For historical accounting, use only previously observed SOL/USD prices or explicitly pause until valid data exist. Mark the asset conservatively with a scenario range when route execution cannot be confirmed, and keep uncertainty distinct from realized P&L.

Acceptance tests: After an extended outage, no 'sell now' value is shown as if executable; historical calculations never consult future SOL prices; unknown prices appear as unknown, with a worst-case scenario, not automatically as the buy price.

Finding 13. The reserve-and-fee model is too static for reliable claims

Severity: P1. **Code evidence:** The model fixes `SUPPLY = 1e9`, `CURVE_K = 30 * 1.073e9`, `POOL_K = 85 * 206.9e6`, and `GRAD_MC_SOL = 410.9`. PumpSwap fees stop updating at 9,820 SOL and remain fixed at 0.95% above that threshold. See lines 3543-3578.

Problem: The code assumes that a price uniquely identifies pool reserves under a fixed invariant. On real pools, reserves and invariant can be altered by prior trades, fee allocation, liquidity operations, and venue-specific state. It does not model an observed transaction route. Current pump.fun official documentation lists additional canonical SOL-pool fee tiers above 9,820 SOL, eventually below 0.95%, so the coded table is incomplete for larger coins. It also describes other supported pair assets, though this bot intentionally filters SOL pairs. Source: pump.fun fee schedule, accessed October 21, 2026.

Required change: For live paper decisions, prefer point-in-time observed on-chain reserves and venue-specific SDK quotes. Version protocol parameters by effective date and pool type, and store the fee schedule used. Calculate executable sell proceeds by quote size, not only mark price. Explicitly include chain transaction costs, priority fees, associated account rent where relevant, routing charges, latency, and failed orders. Never treat a fixed 1% price haircut as an actual paid priority fee.

Acceptance tests: Compare model quotes against authoritative SDK or on-chain reserve calculations over a sample of token ages, sizes, and transitions. Quantify model errors, including at graduation, zero-volume periods, and higher market-cap fee tiers.

Finding 14. Take-profit targets and payoff asymmetry are poorly aligned

Severity: P1. **Code evidence:** The baseline targets `tp = 0.01` (1% *net* profit on cost) while tolerating `sl = 0.90` in spot-price movement. Other variants use 25% TP, a deeper entry, or 30/120-minute timeout. See lines 3579 and 3710-3729.

Problem: A few very large losses can overwhelm many tiny winners. On illiquid curves, fees, fixed haircut, and trade impact create a substantial round-trip drag. A '1% profit target' does **not** mean the price needs to rise only 1%. Using the code's model, a hypothetical 1 SOL round trip at a $7,000 market cap and SOL at $150, with no underlying price move, returns approximately **0.910 SOL**, about a **9.0% loss** from modeled costs and price impact. This is an illustration, not a measured execution quote.

Required change: Estimate the round-trip break-even price **before** trade. Require opportunity size, liquidity, and expected price range to exceed a cost-adjusted edge hurdle. Replace one fixed TP for all setups with exits tailored to trend regime, position age, and deteriorating demand. Test partial profit-taking, trailing exits, and volatility-adjusted timeouts, but do not presume these improve results until measured.

Acceptance tests: Each candidate shows projected round-trip cost at intended size, required move to break even, expected exit cost under stress, and projected risk/reward. Strategies with negative cost-adjusted expectancy are rejected even

if gross win rate is high.

Finding 15. Watchlist and lifecycle design mismatch

Severity: P2. **Code evidence:** Six simultaneous watch slots per bot, two-hour window, one open position per bot, and selection based on earliest qualifying dip. See lines 3709-3729, 3981-4005, 4150-4185.

Problem: One stuck position prevents a bot from using new opportunities for hours or indefinitely. A two-hour window is applied equally to young, thin tokens and graduating tokens, even though their risks and opportunity patterns differ. Watchlist ranking uses a score with dominant name-cluster weight and stale admission-time features.

Required change: Add age/stage-specific strategy classes with separate risk parameters. Treat a held position as competing for capital with available cash and other trades, but do not churn purely because another token's score improved. Introduce expiration, cooldown, and thesis-based abandonment of watched candidates. Scale monitor priority by both risk and opportunity.

Acceptance tests: A dead token cannot monopolize a strategy indefinitely; newly weak watchlist entries are dropped with reasons; a profitable holding is not exited solely because of transient alternative hype.

Finding 16. Graduated coin handling needs independent validation

Severity: P1. **Code evidence:** `stitch()` combines bonding-curve and PumpSwap OHLCV by discarding pool candles timestamped at or before the last curve candle. Graduation is inferred from discovering a pool in DexScreener. See lines 3663-3676, 4063-4083, 4186-4190.

Problem: Graduation can involve overlapping pools, discovery delay, gaps, and data-source timing inconsistencies. Pool discovery is not identical to the exact transition event. Misidentified venue makes simulated reserve, fee, fill and exit prices unreliable. Sparse data around migration can generate false continuity.

Required change: Track a venue transition state machine from authoritative pool and bonding-curve state. Record pool addresses, reserve snapshots, migration event time, and quote availability. Preserve source candles separately, then create a rigorously defined canonical price series. Never fill across an uncertain transition without evidence.

Acceptance tests: Duplicate/overlapping candles do not create phantom trades; graduation gaps pause execution assumptions; a token's curve and pool fills use the correct pool state and fee schedule.

Finding 17. Current risk scoring lacks independent validation

Severity: P1. **Code evidence:** `scoreOf()` gives `(name-match count * 10) + holder contribution up to 10 + social-link presence 3 +`

`organic-score contribution up to 5`. The score primarily manages admission to six watch slots. See lines 3979-4005.

Problem: The largest coefficient rewards mass name duplication, which can be cheap to manipulate. A binary social-link bonus is not evidence of audience. The score lacks age, liquidity, price confirmation, and trade-size impact. No out-of-sample calibration is available in the uploaded file.

Required change: Keep **hard gates** separate from **soft ranking**. Hard gates: sale route, source freshness, security, minimum reliable liquidity, execution capacity. Soft signals: organic growth, order-flow persistence, multi-timeframe structure, narrative evidence, and upside after costs. Optimize/calibrate only on training data and test stability across separate time periods and launch cohorts. Publish feature ablation to show what each feature contributes.

Acceptance tests: Manipulating token-name frequency cannot bypass hard gates. Removing any claimed predictive feature gives a measurable, repeatable performance change or the feature is dropped.

Finding 18. Performance accounting can obscure risk

Severity: P1. **Code evidence:** `acct()` computes maximum drawdown largely from the sequence of realized trades, plus the present value of at most one open position; `renderLab()` highlights the largest balance after at least five closed trades. See lines 4193-4200 and 4880-4895.

Problem: Intratrade drawdown and equity peaks are largely unrepresented. Open positions that cannot exit can inflate balances if their price is stale. Seven bots may have strongly correlated exposure to the same coins, so aggregating seven simulated accounts does not prove seven independent edges. A handful of lucky 1% exits can create a misleading leaderboard. The 'best so far' highlight encourages reading noise as validation.

Required change: Create a chronological **full marked-equity ledger** with separate balances for free SOL, locked positions, estimated recoverable value, unrealized P&L, fees, and quote uncertainty. Report underwater time, max adverse excursion, max favorable excursion, return distribution, concentration, failed-exit frequency, time in market, capital utilization, and drawdown from all observations. Show sample size and uncertainty; do not crown a winner after five trades.

Acceptance tests: A position that rose and then collapsed records a drawdown even before closure. Missing quotes generate bounded or unknown valuation, not artificial stability. Comparative strategy results include shared market exposure and uncertainty.

Finding 19. Reconstructed downtime trades must be labeled hypothetical

Severity: P0. **Code evidence:** The comments and UI say held or watched coins are 'caught up' or 'replayed' after the process was offline. The next run feeds missed finalized candles into the same decision engine and can book historical buys/sells. See lines 3730-3735, 4221-4235, 4945-4952.

Problem: Replaying market data after downtime is valuable **for research**, but it is not equivalent to observing and executing a forward paper decision in real time. An automatic stop cannot execute while the process is offline; retroactively assigning an attractive exit price overstates operational capability.

Required change: Preserve two distinct modes: `retrospective_hypothetical_backtest` and `forward_observed_paper`. When offline, do not silently credit missed transactions to the forward-paper ledger. Mark outages, record potential missed signals, and either resume risk evaluation at a live quote or run a separate clearly labeled reconstruction.

Acceptance tests: An outage spanning a trigger and recovery cannot produce an apparent forward-paper trade executed during the outage. The timeline visibly distinguishes decisions made live, simulated retrospective possibilities, and unexecuted intended exits.

Finding 20. Discovery and data limitations create hidden selection bias

Severity: P1. **Code evidence:** Discovery uses Jupiter `/recent`, follows certain listings for up to 75 minutes, removes those under $4,500 after 20 minutes, requires a pump.fun pool address and SOL quote, and bootstraps name history from trending lists with approximate timestamps. See lines 3853-3931.

Problem: This sampling omits launches missed during downtime or between polling windows, out-of-scope pairs, tokens no longer queryable, and some early failures. `bootstrap()` approximates historical crossings using `c0 + 60`, which is not a verified $7,000 crossing event; this can contaminate meme signals with backdated history. Jupiter defines 'recent' using first pool creation, not necessarily token mint creation. Source: Jupiter, March 26, 2026.

Required change: Maintain an append-only discovery registry of **all** observed qualifying launches and their first-seen time, plus explicit coverage gaps. Do not fabricate crossing event times. Backfill only from archived point-in-time evidence or label the result approximate and exclude it from trading signals. Build a separate historical dataset that includes failed launches and dead tokens.

Acceptance tests: Synthetic bootstrap items cannot masquerade as confirmed historical crossings. Statistics disclose scanner uptime and eligible population, including the proportion without available candles or audits.

C. Redesigned strategy architecture

The objective is **not** to make the model sound more intelligent. The objective is to make each action causally defensible, appropriately priced, and measurable.

C1. A six-layer decision system

Layer 1, provenance and observation: Ingest token metadata, price and transaction events, pool state, route quotes, audits, and optional social evidence. For each datum preserve `event_time`, `first_seen_at`, `received_at`, `source`, `source_id`, `age`, and `confidence`. No future value can enter a historical decision.

Layer 2, hard eligibility: Reject unverified token identity, unusable sell routes, unknown critical security facts, unacceptably concentrated ownership, unsafe transfer settings, stale pricing, and inadequate depth for intended stake. Unknown is not 'safe'. Do not replace explicit security rules with AI sentiment.

Layer 3, market regime and setup: Classify token stage, trend, and market microstructure. A freshly launched curve token requires different evidence and timing from a token that has graduated and traded for hours. Detect momentum ignition, continuation, confirmed pullback recovery, and exhaustion. Reject setups without estimated post-cost opportunity.

Layer 4, narrative and social research: Identify a token by mint, not ticker. Track the evolution of a specific narrative and independent attention, verify whether cited accounts actually discuss the token, separate genuine discussion from coordinated farming, and represent uncertainty. Avoid using scraped engagement as ground truth. Use language models for summarization and contradiction detection, not as the sole trading authority.

Layer 5, position control: Record the reason for entry, expected trading horizon, what must remain true, invalidation conditions, quote-based liquidation value, and risk budget. Reassess continuously. Handle exits with a priority order: critical security and route events, unacceptable execution/liquidity risk, material thesis break, time expiry, trailing-profit regime change, discretionary target.

Layer 6, execution and accountability: Submit only simulated orders after the supporting data were observed. Apply venue quotes or a clearly bounded execution model, latency, costs, possible rejection, fill confirmation, and a full portfolio ledger. Track candidates rejected and opportunities missed, not only fills.

C2. Candidate strategies should compete, not collapse into one monster model

Strategy A, Launch Momentum: Extremely young, curve-phase tokens, high event frequency, short time-to-live, high emphasis on organic buyer growth,

security, and immediate exit access. Do not use slow social analysis as a blocking requirement where the narrative cannot be validated in time.

Strategy B, Confirmed Pullback Recovery: Enter only after a strong prior move, a controllable pullback, evidence that selling is subsiding, and renewed demand. A raw -40% candle is **not** enough. Invalidation is a failed reclaim or reacceleration of seller pressure, not a 90% collapse.

Strategy C, Post-Graduation Narrative Momentum: Tokens with established pools and stronger liquidity; more room for social verification and trend continuation; larger time windows and trend-based exits. This is a separate hypothesis from 60-second launch sniping.

Strategy D, No Trade: Cash in SOL is an explicit and often correct action. Evaluate against holding SOL and against doing nothing. The bot should not force activity merely because a watchlist token reached a percentage trigger.

Each strategy gets distinct configuration, tests, exposure budget, and performance reporting. Preserve the existing U, U2, U30, T25, D55, GRN, and STR variants as **legacy baselines**, not as evidence-backed final models.

C3. Entry contract for every trade

Every intended buy must record:

- Token mint and canonical venue, first observation, token age, graduation stage.
- Exact features available at decision time and their timestamp/freshness.
- Security gate results, including unknown fields and conflicts.
- Setup type and what specifically confirms it.
- Evidence of independent buyers and direction of actual *notional* order flow.
- Current reserve depth and realistic quote for the intended stake.
- Estimated entry costs, executable break-even level, slippage sensitivity.
- Expected holding horizon, risk budget, invalidation criteria, and emergency response.
- Why this coin is preferable to other candidates **at this decision time**.
- Whether any social narrative evidence is independently verified, suspected, or unavailable.

Gate sequence: identity -> safety -> sellability -> timely data -> market structure -> demand confirmation -> expected edge after costs -> capital availability -> submit paper order. A later gain cannot be used as evidence that the earlier entry was well justified.

C4. Exit contract, including 'coin dying'

Do not implement `if price red -> sell`. Design a structured exit hierarchy:

1. **Emergency:** verified malicious contract or authority change, liquidity disappearance, serious route failure, validated creator/cluster exit event, or unacceptably large estimated sell impact. Attempt exit if possible; record an unfilled order if impossible. These events should not depend on a normal stop.
2. **Thesis break:** material, corroborated deterioration. For example, failed reclaim plus sharply negative net buy value and shrinking distinct buyers. Apply confidence and persistence, since one transient candle is noisy.
3. **Risk-limit exit:** position loss or current worst-case recoverable value breaches the predefined risk budget. Use actual size-adjusted sell estimates.
4. **Time and stagnation exit:** the expected setup has not developed within its declared horizon; reconsider cost of keeping capital tied up. This must be trade-specific, not always 30 minutes or always 2 hours.
5. **Profit-protection exit:** trend weakens after a favorable move, quote liquidity deteriorates, or a trailing regime threshold is reached. Explore partial sales only where fees and impact do not make them uneconomic.
6. **Target exit:** if a strategy truly benefits from a fixed target after costs. Never assume 1% net is universally suitable.

Suggested starting feature families for research, not fixed thresholds:

Family	Inputs to test	Why it matters	Common failure mode
Market structure	Failed highs, reclaim failure, volatility-normalized drawdown	Separates correction from breakdown	Noisy bars trigger churn
Real demand	Unique buyers, net SOL flow, size-weighted buy/sell	Detects withdrawal of demand	Bot and wash activity
Liquidity and sellability	Quote depth, sale impact, route count, reserve drift	Determines recoverable value	Displayed price misleads
Holder and developer behavior	Cluster distribution, concentration changes, known related wallets	Detects coordinated selling	False positives from innocent linked wallets

Family	Inputs to test	Why it matters	Common failure mode
Social attention	Independent mentions, velocity, source credibility, narrative continuity	Measures attention rather than ticker copies	Paid amplification and misinformation
Time and lifecycle	Minutes since launch, curve progress, migration, time since demand peak	Makes thresholds stage-specific	Rigid global time limits

Avoid double-counting: price drop, market-cap drop, and drawdown may all describe the same event. Transaction count and buy volume are related but not interchangeable. A strong risk score should reflect multiple independent pieces of evidence rather than adding correlated penalties repeatedly.

C5. Research engine for X and community hype

Research should answer specific questions, not issue an opaque 'viral: yes' label. For shortlisted candidates, ask:

1. What is the claimed narrative, and does it refer to this **exact mint**, not a lookalike ticker?
2. When did this narrative begin, and were mentions visible before the trade signal?
3. Are diverse, credible independent accounts discussing it, or mostly one cluster of copy-paste promotions?
4. Is attention accelerating relative to that token's own history and age, not just a large raw count?
5. What percentage of mentions include clear token-specific identifiers versus only a generic meme word?
6. Is there verifiable evidence of an official partnership or celebrity association, or only unauthenticated claims?
7. Does stronger attention correlate with increased distinct buyers, usable liquidity, and sustained trade flow?
8. Is sentiment still strong after a sharp pullback, or has participation ceased?

Data policy: Respect access restrictions and source terms; cache economically; timestamp first sighting; store links and excerpts or content identifiers where permitted; give each source freshness/credibility. Rate-limit external research. Set `social_status = unavailable` when paid APIs or permissions are absent.

An optional AI summarizer should return **claim, supporting source, contradicting source, observation time, and confidence** and may never fabricate missing counts.

Important product idea: Add a `Narrative Dossier` button for the user, showing verifiable posts and a clear `evidence incomplete` warning. The dossier is for understanding trade decisions, not a claim that the coin is legitimate or that social posts predict returns.

C6. On-chain risk and sell-route realism

Use Jupiter token audits and organic metrics for initial filtering, but confirm critical actions using pool state, token-program authority, current sell quote, and wallet transaction evidence from appropriate authorized infrastructure. Distinguish mint/freeze authorities from creator trading behavior. A token may use standard SPL token code and still be manipulated through trading activity. Cluster analysis must document false-positive risk. A failed sell attempt is part of the outcome, not a case to omit from the dataset.

Quote-based size selection: Position size should respond to quoted exit capacity and worst-case slippage. Do not use a fixed 1 SOL order if doing so consumes excessive available depth. Capital allocation must also obey a total portfolio risk budget. The correct size is often zero.

C7. Robust market data architecture

Use a structured, append-only event log rather than mutating one rolling object and only saving recent winners. Suggested records:

- `TokenDiscovered(tokenMint, observedAt, pair, feed, payloadHash)`
- `CrossingObserved(tokenMint, observedAt, level, source)`
- `AuditReceived(tokenMint, requestedAt, receivedAt, source, fields, status)`
- `MarketSnapshot(tokenMint, exchangeTime, receivedAt, quote, reserves, liquidity, flow, confidence)`
- `SocialObservation(tokenMint, firstSeenAt, sourceId, claim, verification)`
- `EligibilityDecided(tokenMint, at, featureVersions, gates, reasonCodes)`
- `SignalGenerated(strategyId, tokenMint, at, direction, thesis, confidence)`
- `OrderRequested(strategyId, tokenMint, at, quoteId, size, riskBudget)`
- `OrderFilled / OrderFailed(at, fee, slippage, route, explanation)`
- `PositionReviewed(at, thesisStatus, risk, action)`
- `PositionClosed(at, realizedProceeds, fees, exitReason)`
- `CoverageGap(start, end, source, consequence)`

A replay engine should rebuild portfolio state by processing events **in actual observation order**. A later API response may update current knowledge, but cannot alter earlier paper decisions. Maintain separate replay and forward-paper ledgers.

D. Quantitative evaluation plan

D1. Research dataset

Collect all observed launches in the declared scope, not only eventual winners. Preserve launches without a crossing, early deaths, failed audits, absence of valid quotes, rugged liquidity, failed graduation, successful graduation, sustained winners, and tokens with apparent wash activity. Retain data-coverage gaps. Collect base currency SOL and a contemporaneously observed USD exchange rate, so returns can be reported in SOL and USD without accidentally conflating them.

D2. Experiments in a controlled order

Experiment	Question	Controlled change
E0	How much of the current P&L survives rigorous timing?	Keep original decisions, remove retroactive eligibility and same-bar fills
E1	Does credible market flow improve entries?	Add true buy/sell notional and distinct buyer persistence
E2	Does explicit downside invalidation help?	Keep entries fixed, add deterioration and time exits
E3	Does realistic execution destroy apparent edge?	Quote-based slippage, latency, failures, fee sensitivity
E4	Does social research have predictive value?	Compare otherwise-identical strategies with and without timestamped social features
E5	Which setup family survives unseen periods?	Separate launch momentum, pullback recovery, graduated momentum

Experiment	Question	Controlled change
E6	How sensitive is the result to operational failures?	API outages, missing candles, graduation ambiguity, stale quote, delayed sell

Run E0 before E1. If the original paper returns depend materially on backdating or unexecutable touches, strategy optimization on those returns is invalid.

D3. Validation methodology

Use chronological development and validation segments, then **untouched test dates**. Embargo overlapping launch cohorts where leakage is possible. Walk forward with frozen parameters. Group closely related meme narratives and wallets when calculating uncertainty, because many signals are not independent. Use bootstrap intervals over days/cohorts, not naive independent-trade intervals. Do not tune on the test set after seeing disappointing results.

Compare against: original U/U2/U30/T25/D55/GRN/STR, no-trade SOL holding, simple timed exit on identical entries, simple entry with no social factor, random qualified candidate choice at the same time, and a market-cap/volume baseline. Separate alpha due to picking coins from alpha due to choosing entry/exit times.

Report at minimum: number of launches observed, fraction passing gates, entries, closed trades, still-open distressed positions, net return in SOL and USD, realistic fees, slippage, route failures, hit rate, payoff ratio, profit factor, post-cost expectancy, max marked-to-market drawdown, worst-day loss, time underwater, realized and unrealized P&L, opportunity cost, exposure by lifecycle stage, and maximum estimated liquidation loss. Always provide sample size and uncertainty. For live forward paper, report **decision age** and **quote-to-fill age**.

D4. Sensitivity testing

Sweep fees, extra slippage, order submission latency, price gaps, missed triggers, stale-price windows, partial fill probability, capital per position, and simultaneous coin correlation. Repeat with more conservative assumptions than the default. A promising strategy should not become catastrophically unprofitable after one realistic adjustment.

D5. A realistic promotion rule

A new strategy may replace the old one only if: it passes all P0 correctness tests; performs at least as well as the original on a causally valid simulation with realistic costs; demonstrates stable results in untouched forward time windows;

has no hidden drawdown or no-route failure problem; and is understandable enough to diagnose. If no approach passes, **the correct output is a research-grade simulator with no claimed edge**.

E. Implementation roadmap with concrete deliverables

Phase 0: Freeze baseline and protect data

Tasks: Export existing journal, configuration, and source hashes. Save U/U2/U30/T25/D55/GRN/STR outcomes as historical baselines, explicitly marked as using legacy assumptions. Add immutable experiment identifiers. Set the default mode to paper only; no private key, wallet connection, or real orders.

Deliverable: Reproducible snapshot, source manifest, old versus new namespace, test harness. **Exit criterion:** Original data remain readable and unchanged.

Phase 1: Repair causal correctness (P0, do this before optimization)

Tasks: Introduce `eligible_at`; bar-close next-event fill; no future SOL conversion; explicit quote age; stop backdating during downtime; mark unavailable pricing; distinguish requested from completed paper orders. Add unit tests from Findings 01, 02, 12 and 19.

Deliverable: Deterministic timestamp-aware event engine and regression suite. **Exit criterion:** All paper trades can be justified from data known at or before their submission timestamps; no retroactive trade is labeled live.

Phase 2: Repair survival controls and real-world execution

Tasks: Reject unreachable stops; implement position risk budgets, default finite thesis horizon, emergency sellability states, quote-based proceeds, current protocol fee tiers and reserve-state inputs, failure modeling and a measured equity ledger. Add a kill switch for stale critical data and a priority queue for held positions.

Deliverable: Credible risk and execution simulator. **Exit criterion:** No bot remains confidently valued on old entry prices; extreme coin deaths show losses or unpriceable positions rather than phantom protection.

Phase 3: Add market intelligence before social intelligence

Tasks: Implement feature snapshots, flow and liquidity indicators, separate entry strategies, candidate ranking, independent deterioration exits, confidence-aware guards, and decision reasons. Run feature ablation and realistic baselines.

Deliverable: A rules-based, interpretable V2 with measurable market-structure intelligence. **Exit criterion:** It distinguishes a confirmed recovery

from a collapse at the same percent drawdown, and can justify every trade.

Phase 4: Add verified narrative research

Tasks: Connect only approved APIs, mint-resolve social references, score independent mention velocity and narrative persistence, attach sources and timestamps, cache economically, prevent unsupported assertions, add dashboard dossier. Run the social-feature ablation experiment.

Deliverable: Evidence-backed social layer with `verified / uncertain / unavailable` signals. **Exit criterion:** Social inputs demonstrate incremental test-set value or remain strictly informational.

Phase 5: Validation, operations, and dashboard

Tasks: Collect unbiased history, maintain forward shadow trading, run latency/stress tests, compare simultaneous strategies, audit dashboard timestamps, publish uncertainty and failure rates. Preserve source core sharing across browser and server, but audit the separate GitHub Actions deployment for schedule gaps and journal races.

Deliverable: Updated application, source diff, tests, repeatable analysis, benchmark report, and remaining-risk register. **Exit criterion:** Meets promotion rule in Section D5, otherwise retain legacy and V2 as labeled experiments.

F. Suggested automated test suite

ID	Scenario	Required outcome
T01	Check received five minutes after a historical dip	No retroactive forward-paper entry
T02	Green candle known only at minute close	Earliest fill is a subsequent eligible event
T03	One-tick wick touches a trigger with no executable quote	No guaranteed fill
T04	Low-cap position has stop below curve floor	Entry rejected or realistic alternate risk defined
T05	Coin falls sharply, buyers vanish, trend fails	Timely thesis-invalidating exit attempt

ID	Scenario	Required outcome
T06	Healthy pullback with renewed buying	Not automatically sold merely because candle is red
T07	Pool or route disappears	Unfilled exit is logged; value marked uncertain/distressed
T08	Social mentions are copies from one account group	No automatic high-trust hype status
T09	Celebrity or partner claim lacks verifiable source	Unverified label, not endorsement
T10	Developer balance and top-holders fields missing	Unknown, no fake zero or pass
T11	Price feed stale; prior close still available	Stale mark clearly identified; new entries halted
T12	SOL/USD minute missing	No search into future data
T13	Token graduates between curve and pool candles	Correct state transition or execution paused
T14	Server offline during target/stop	Hypothetical replay kept separate from live paper ledger
T15	Same event arrives twice; server restarts	Idempotent order and journal handling
T16	Strong winner trends far above fixed TP	Alternative trailing exit assessed without hindsight
T17	Size doubled in thin liquidity	Quote impact rises and size may be rejected
T18	Winning trade has major intratrade adverse excursion	Drawdown and excursion recorded
T19	Multiple copycat tokens share a name	Name repetition cannot satisfy ‘verified social’
T20	API returns a schema change or unexpected missing fields	Safe failure with logged reason

G. Dashboard, observability, and user-facing trust

Top status bar: `Forward paper`, `Historical replay`, or `Incomplete data`, never ambiguous. Show scanner uptime, current source latency, most recently validated quote, and whether paper trading is actively evaluating signals.

Watchlist: Exact mint and venue; stage; age; last market event; live liquidity/quote size; organic buyer trajectory; security gate status; social evidence; reason candidate is not yet bought; time since last validation. Replace 'the first to dip wins' with 'ranked eligible signals with evidence and net-cost estimates'.

Held position card: Entry thesis; current thesis health; largest current risk; age since last verified sell quote; recoverable SOL at intended full position size; worst-case quote; realized/unrealized and liquidation costs; reason to hold; most likely reason to exit. Visibly distinguish `not safe to sell` from `not enough current data to confirm sellability`.

Trade audit: Signal timestamp; actual decision timestamp; audit timestamps; order submission; fill or failure; quote route; nominal versus realized price; fee breakdown; last changed evidence; exit reason. User should be able to reconstruct exactly how the bot arrived at each decision.

Strategy comparison: Show equity based on all marked observations, not only closing trades. Include open-loss tail, minimum number of observations, and confidence intervals. No automatic winner badge after five trades. Display an independent baseline of simply holding SOL.

H. Engineering notes and integration boundaries

- The uploaded ZIP contains a browser `index.html`, manifest and icons. It does **not** contain deployed server workers, tests, complete API schemas, private credentials, a trade-event archive, or raw backtest fixtures. Audit those separately if available before claiming production parity.
- The shared `SNIPER` and `SNIPER_CORE` definitions are embedded directly in the HTML. Preserve compatibility or extract them into separately tested modules bundled into the page; do not casually create code drift between server and browser.
- The current client expects the public repo `AliMallick2008/meme-sniper`, branch `data`, for server state. A read-only inspection of deployment configuration should be part of operational validation; this source review alone does not prove how reliably scheduled server jobs run.
- Do not use language-model reasoning as an opaque trade generator. LLMs are useful to summarize news, compare claims and attach citations, while event-driven deterministic safeguards control risk, eligibility and execution.
- Do not interpret high win rate as profitability. A system winning 90 tiny trades and losing 10 very large ones can lose money after costs.

- A fixed pre-trade fee/slippage assumption is useful for screening but cannot certify route availability or final proceeds.
- The time cost and price of X data matter. Building a sophisticated social pipeline that finishes after an extremely fast pump is not useful; choose strategy horizons accordingly.
- Do not include real-money wallet functionality in this upgrade. The purpose is to understand whether a strategy has any robust edge before exposing capital.

I. Agent-ready implementation brief

Objective: Overhaul the existing Meme Sniper v10.1 in the supplied BTC Paper Trading Lab source code. Do not merely adjust the dip or stop constants. First correct P0 timing, point-in-time data, non-executable candle fills, and stale valuation so that the paper results are causally valid. Then implement stage-specific entry strategies, independent multi-signal deterioration exits, reliable liquidity and sale-route checks, explicit unknown-risk handling, and an interpretable decision log. Add optional source-linked social and narrative intelligence only after basic market/risk features work, and validate its incremental benefit through feature ablations. Keep the entire project paper-only.

Sequence is mandatory: (0) preserve baseline and journal, (1) implement time-valid event and paper-order ledger, (2) repair stops, exits and executable quote model, (3) implement market intelligence and comparative strategy experiments, (4) integrate verified narrative sources, (5) run realistic forward and out-of-sample validation, (6) update dashboard and write final report. Treat findings 01 through 04, 09, 12, 13, and 19 as gating defects. Do not optimize on legacy headline P&L or use information that arrived after a trade's supposed execution.

Evidence required: Produce a source-code diff, tests for T01 through T20, an old/new reconciliation of each legacy trade that changes after causal fixes, a list of integrated API endpoints and failure modes, a point-in-time dataset provenance report, an execution cost breakdown, results against all legacy baseline bots plus holding SOL, and an honest statement of what was not verifiable. If no robust improvement remains after realistic costs and unseen data, report that explicitly. Do not claim the bot 'knows' a coin will pump, can automatically recognize every rug, or is profitable without evidence.

J. Research sources and documentation

1. **Pump.fun official fees.** Canonical pool fee tiers and bonding-curve trading fees, including tiers above 9,820 SOL: https://pump.fun/docs/fees
2. **Pump.fun bonding curve explanation.** Constant-product pricing and graduation concepts: https://pump.fun/docs/bonding-curve
3. **DexScreener API reference.** Supported token/pair fields, recent volume and transaction count, liquidity, rate limits: https://docs.dexscreener.com/api/reference
4. **Jupiter, organic scoring.** March 26, 2026 explanation of organic scoring, limitations for fresh tokens, and `/recent` first-pool semantics: https://developers.jup.ag/blog/what-is-organic-score
5. **Jupiter, source price integrity.** March 20, 2026 explanation of liquidity, wash-trading and price heuristics: https://developers.jup.ag/blog/how-jupiter-prices-a-token
6. **USENIX Security 2026, Mongardini and Mei.** *A Midsummer Meme's Dream: Investigating Market Manipulations in the Meme Coin Ecosystem.* Empirical evidence of wash activity and price inflation in cross-chain meme coins: https://www.usenix.org/conference/usenixsecurity26/presentation/mongardini
7. **Jupiter swap design reference.** Quotes, slippage and route availability are specific to tokens and sizes: https://developers.jup.ag/docs/guides/how-to-build-a-custom-swap-with-metis

Research caution: These sources motivate risk controls and data integration; they **do not** demonstrate that the recommended V2 strategy is profitable. Any new thresholds, rankings or social features remain hypotheses until tested.

Final conclusion

The best first improvement is not 'sell sooner' in isolation. It is to make the simulator honest about **what it knew**, **when it knew it**, **what it could actually trade**, and **why it still owns a token**. Without those foundations, improvements to hype analysis or AI scoring can simply make an unreliable backtest more complicated. With them, the project can become a rigorous platform for comparing fast launch trading, confirmed pullbacks, and slower narrative momentum, including the possibility that none of them overcomes real trading costs.